

Dobot Magician

Befehlskette - Zusammenfassung der Versionen 3.3 bis 3.3.4

Vom seriell angeschlossenen ESP32 über die COM/TCP-Umschaltung bis zur Verarbeitung dauerhafter Sensorwerte und zur vollständigen Vorabprüfung der Befehlskette.

Version	Schwerpunkt	Kernaussage
3.3	ESP32 über COM	Tastatur und ESP32 verwenden dieselbe Steuerbefehls-Queue.
3.3.1	COM oder TCP	Echter ESP32 oder lokaler TCP-Simulator.
3.3.2	Ablaufsteuerung	Marken, Sprünge, Warten, HOME und Geschwindigkeit.
3.3.3	ESP32-Werte	Sensor- und Zustandswerte werden dauerhaft gespeichert.
3.3.4	Prüfen und Rückmelden	Vorabprüfung, Vergleiche und Zustandsmeldungen.

Merksatz: 3.3 = ESP32 über COM; 3.3.1 = COM oder TCP; 3.3.2 = Ablaufsteuerung; 3.3.3 = ESP32-Werte; 3.3.4 = Prüfen und Rückmelden.

Version 3.3

Schwerpunkt: ESP32 als zweite Steuerquelle über die serielle COM-Verbindung

Die Befehlskette wird erstmals nicht nur über die PC-Tastatur, sondern zusätzlich durch einen ESP32 gesteuert. Beide Quellen liefern ihre Befehle an dieselbe Queue. Nur die Hauptschleife greift auf die Dobot-API zu.

Änderungen und Erweiterungen

- Steuerbefehle des ESP32: PAUSE, WEITER, HALT und STATUS.
- Ein eigener Empfangsthread liest die COM-Schnittstelle, ohne die Dobot-Ausführung zu blockieren.
- Die Quelle wird mitgeführt, beispielsweise ESP32-COM oder Tastatur.
- Handshake und einfache Antworten wie ESP32_BEREIT, PC_BEREIT, PING und PONG werden unterstützt.
- Die PC-Tastatur bleibt auch bei fehlender ESP32-Verbindung nutzbar.

Typische Beispiele

ESP32-COM -> Steuerbefehls-Queue -> Hauptschleife -> Dobot

Tastatur -> Steuerbefehls-Queue -> Hauptschleife -> Dobot

Bekannte Grenzen

- Allgemeine Ereignisse und Sensorwerte werden noch nicht dauerhaft gespeichert.
- Virtuelle COM-Ports für den Simulator führten unter Windows zu Treiberproblemen.

Bedeutung für die Entwicklung

Version 3.3 schafft die gemeinsame Architektur für mehrere Steuerquellen.

Version 3.3.1

Schwerpunkt: Umschaltung zwischen serieller COM-Verbindung und TCP

Der echte ESP32 kann weiterhin seriell angesprochen werden. Alternativ ersetzt ein TCP-Server den ESP32 und ermöglicht Tests ohne Mikrocontroller und ohne virtuelle COM-Port-Treiber.

Änderungen und Erweiterungen

- Auswahl über COM_MODUS = "serial" oder COM_MODUS = "tcp".
- Der serielle Modus verwendet COM-Port, Baudrate und Verbindungstimeout.
- Der TCP-Modus verwendet Hostadresse, TCP-Port und Verbindungstimeout.
- Der Simulator kann lokal unter 127.0.0.1 laufen.
- Die Befehlskettenlogik bleibt vom Kommunikationsweg unabhängig.

Typische Beispiele

```
COM_MODUS = "serial" # echter ESP32 über COM
```

```
COM_MODUS = "tcp" # ESP-Simulator über TCP
```

Bekannte Grenzen

- Gleichnamige Module in mehreren Projektordnern konnten in Thonny Shadowing-Warnungen erzeugen.
- Deshalb wurden spätere Dateien konsequent mit eindeutigen Versionsnamen versehen.

Bedeutung für die Entwicklung

Version 3.3.1 macht Kommunikation und Befehlskettenlogik austauschbar.

Version 3.3.2

Schwerpunkt: Marken, Sprünge, Warten, HOME und Geschwindigkeitsänderung

Die lineare Befehlsfolge wird zu einem kleinen Ablaufprogramm. Marken und Sprünge erlauben Wiederholungen, während Meldungen des ESP32 gezielt abgewartet werden können.

Änderungen und Erweiterungen

- Neue Marke: ("marke", "START").
- Neuer Sprung: ("gehe_zu_befehl", "START").
- Warten auf eine einmalige Meldung mit warte_bis.
- HOME-Fahrt als eigener Befehl der Befehlskette.
- Geschwindigkeit und Beschleunigung können im Ablauf geändert werden.
- Doppelte Marken und unbekannte Sprungziele werden erkannt.
- Programmende und Halt werden dem ESP32 beziehungsweise Simulator gemeldet.

Typische Beispiele

("warte_bis", "FREIGABE", "Warte auf Freigabe", None)
("geschwindigkeit", 40, 40, "Geschwindigkeit auf 40 % setzen", 0)
("gehe_zu_befehl", "AUF_FREIGABE_WARTEN")

Bekannte Grenzen

- Einmalige Meldungen werden aus der Queue entnommen und sind danach verbraucht.
- Für Rücksprünge werden Hochsprachenbefehle schrittweise statt vollständig vorab abgearbeitet.

Bedeutung für die Entwicklung

Version 3.3.2 führt die eigentliche Ablaufsteuerung innerhalb der Befehlskette ein.

Version 3.3.3

Schwerpunkt: Dauerhafte Sensor- und Zustandswerte des ESP32

Der Kommunikationsthread speichert nun Werte unabhängig vom Ablauf. Die Befehlskette kann diese Werte später anzeigen, abwarten oder für Entscheidungen verwenden.

Änderungen und Erweiterungen

- Neues Format: WERT;Name;Inhalt.
- Beispiele: WERT;POSITION;POS_A, WERT;TASTER;1 und WERT;TEMPERATUR;23.7.
- Neue Befehle: wert_anzeigen, warte_bis_wert und wenn_wert.
- Ein Wert bleibt gespeichert, bis ein neuer Wert mit demselben Namen eintrifft.
- Steuerbefehle, einmalige Meldungen und dauerhafte Werte werden getrennt verarbeitet.

Typische Beispiele

```
("wert_anzeigen", "TEMPERATUR", "Aktuelle Temperatur")
```

```
("warte_bis_wert", "TASTER", 1, "Warte auf Taster", None)
```

```
("wenn_wert", "POSITION", "POS_A", WAHR_BEFEHL, FALSCH_BEFEHL)
```

Bekannte Grenzen

- Die erste Fassung von wenn_wert war hauptsächlich für Gleichheitsprüfungen ausgelegt.
- Erweiterte Vergleichsoperatoren folgten in Version 3.3.4.

Bedeutung für die Entwicklung

Version 3.3.3 verbindet den Roboterablauf mit dauerhaften Sensor- und Anlagenzuständen.

Version 3.3.4

Schwerpunkt: Vorabprüfung, Vergleichsoperatoren und Zustandsmeldungen

Die gesamte Befehlskette wird geprüft, bevor der Dobot verbunden und gestartet wird. Bedingungen können Texte und Zahlen vergleichen. Gleichzeitig erhält der ESP32 ausführliche Meldungen über den aktuellen Programmzustand.

Änderungen und Erweiterungen

- Prüfung von Befehlsnamen, Parameterzahlen, Wertebereichen, Marken und Sprungzielen.
- Auch eingebettete Befehle und Vergleichsoperatoren werden geprüft.
- Operatoren: ==, !=, <, <=, > und >=.
- Die Operatoren stehen in wenn_wert und warte_bis_wert zur Verfügung.
- Der ESP32 erhält Meldungen über Start, Befehle, Wartezustände, Bedingungen, Sprünge und Programmende.
- Diese Rückmeldungen können LEDs, Displays oder andere Anzeigen steuern.

Typische Beispiele

```
("wenn_wert", "TEMPERATUR", ">=", 30, WAHR_BEFEHL, FALSCH_BEFEHL)
```

```
("warte_bis_wert", "TASTER", "==", 1, "Warte auf Taster", None)
```

```
PROGRAMM_GESTARTET;3.3.4
```

```
WARTE_AUF;WERT;TASTER;==;1
```

```
PROGRAMM_BEENDET;NORMAL
```

Bekannte Grenzen

- Mechanisch sichere Koordinaten und Abläufe bleiben Aufgabe des Programmentwurfs.
- Das direkte Senden beliebiger Befehle aus der Befehlskette an den ESP32 war noch nicht enthalten.

Bedeutung für die Entwicklung

Version 3.3.4 erhöht Sicherheit, Nachvollziehbarkeit und die Zusammenarbeit zwischen PC und ESP32.

Entwicklungslinie: anbinden - Kommunikationsweg wählen - Ablauf steuern - Werte verarbeiten - vorab prüfen und rückmelden.